

Theory of Automaton

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

- (1) **Theory of Computer Science: Automata, Languages and Computation**
by K.L.P. Mishra, N. Chandrasekaran

Please refer to the above books for more details.

Contents

1	Finite Automata	5
1.1	Introduction to Finite Automaton	5
1.1.1	Equivalence with NDFA	5
1.2	Identities for Regular Expressions	5
1.3	Arden's Theorem	6
1.3.1	Transition System Containing Λ -moves	6
1.4	Finding the Regular Expression Recognized by a Transition System . . .	7
1.5	Equivalence of Two Finite Automata	8
2	Minimization of Finite Automata	9
2.1	Minimization Definitions	9
2.2	Example: Construct the Minimum State Automaton	9
3	Formal Languages	11
3.1	Introduction	11
3.1.1	Grammar Definition	11
3.2	Chomsky Classification of Languages	11
3.3	Relation Between Languages	12
3.4	Construction of a Transition System Accepting a Regular Grammar . . .	12
4	Context-Free Languages	14
4.1	Property of Languages That Are Regular	14
4.2	Ambiguity in Context-Free Grammars	14
4.2.1	Construction of Reduced Grammars	14
5	Elimination of Null Productions	15
5.1	Example: Find a Reduced Grammar Equivalent to the Given Grammar .	15
5.2	Lemma 1: Deleting a Variable B Appearing as the First Symbol on the R.H.S.	16
5.3	Lemma 2: Eliminating Left Recursion	16
6	Greibach Normal Form	17
6.1	Definition	17
7	Chomsky Normal Form for Context-Free Grammars	18
7.1	Reduction to Chomsky Normal Form	18
7.2	Remove A -production from the following grammar	19

8	Pushdown Automata	21
8.1	Introduction	21
8.1.1	Pushdown Automaton for $(a^n b^n \mid n \geq 1)$ (Initial Pushdown Symbol)	21
8.2	Infinite Context-Free Languages	21
8.3	Determinism and Parsing	21
9	Turing Machines and Recursive Enumerable Languages	23
9.1	Turing Machines	23
9.2	Linear Bounded Automaton	23
10	Recursive and Recursive Enumerable Languages	24
10.1	Definitions	24
11	Undecidability	25
11.1	Halting Problem	25
11.2	Reduction	25
11.3	Unsolvable Problems about Grammars	26
12	Some Important Results	27
12.1	Regular Expression Results	27
13	NP Completeness	28
13.1	Tractable Problems	28
13.2	Intractable Problems	28
13.3	P-Class Problems	28
13.4	NP-Class Problems	28
13.5	NP-Hard Problems	29
13.5.1	P-class Problems	29
13.6	NP-Complete Problems	29
13.7	Examples of NP-Complete Problems	30

Chapter 1

Finite Automata

1.1 Introduction to Finite Automaton

Definition 1.1.1 (Finite Automaton). The finite automaton has complete absence of memory outside its fixed control processor. It delivers an output at all except on an indication of whether or not the input is considered acceptable.

A deterministic finite state transducer is a device much like a DFA except that its purpose is not to accept strings or languages but to transform input strings into output strings.

1.1.1 Equivalence with NDFA

For every NDFA, there exists a DFA which simulates the behavior of NDFA. NDFA is a more general machine without being more powerful.

If an NDFA has n states, the corresponding finite automaton has 2^n states. However, we need not construct a DFA for all these 2^n states, but only for those states reachable from (q_0) .

Two automata are considered to be equivalent if they accept the same language, even though they may use different methods to do so.

The class of languages accepted by finite automata is the same as the class of regular languages—those that can be described by regular expressions.

Definition 1.1.2 (Regular Language). A language is regular if and only if it is accepted by a finite state automaton.

For any language L :

$$\emptyset L = L \emptyset = \emptyset$$

1.2 Identities for Regular Expressions

Let Φ , R , and S be regular expressions. The following identities hold:

- (1) $\Phi + R = R$
- (2) $\emptyset R = R \emptyset = \emptyset$
- (3) $\Lambda R = R \Lambda = R$

- (4) $\Lambda^* = \Lambda$ and $\Phi^* = \Lambda$
- (5) $R + R = R$
- (6) $R^*R^* = R^*$
- (7) $(R^*)^* = R^*$
- (8) $(R^*)^+ = R^*R^* = R^* = \Lambda + R^*R$
- (9) $(PQ)^*P = P(QP)^*$
- (10) $(P + Q)R = PR + QR$
- (11) $R(P + Q) = RP + RQ$
- (12) and $R(P + Q) = RP + RQ$

1.3 Arden's Theorem

Theorem 1.3.1 (Arden's Theorem). Let P and Q be two regular expressions over Σ . If P does not contain Λ , then the following equation in R :

$$R = Q + RP$$

has a unique solution given by:

$$R = QP^*$$

1.3.1 Transition System Containing Λ -moves

Suppose we want to replace a Λ -move from vertex v_1 to vertex v_2 . Then we proceed as follows:

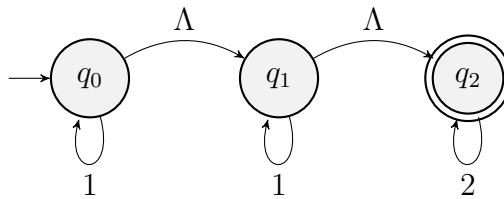
Step 1: Find all edges starting from v_2 .

Step 2: Duplicate all those edges starting from v_1 without changing the edge labels.

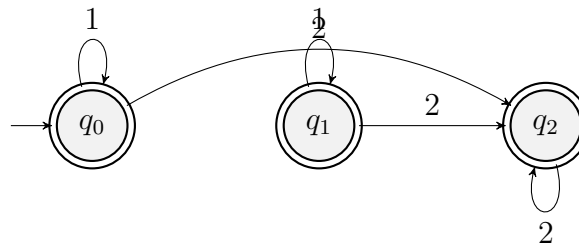
Step 3: If v_1 is an initial state, make v_2 also an initial state.

Step 4: If v_2 is a final state, make v_1 as the final state.

Example 1.3.1. Consider the following transition system:



After removing Λ -moves:

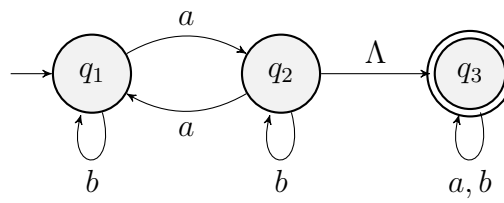


1.4 Finding the Regular Expression Recognized by a Transition System

The following assumptions are made regarding the transition system:

- (i) The transition graph does not have Λ -moves.
- (ii) It has only one initial state, say q_i .

Example 1.4.1. Consider the following DFA:



The equations for each state:

$$\begin{aligned}
 q_1 &= q_1 a + q_2 b + \Lambda \\
 q_2 &= q_2 a + q_2 b + q_3 a \\
 q_3 &= q_2 a (b + aa)^*
 \end{aligned}$$

Placing value of q_1 in equation (i):

$$\begin{aligned}
 q_2 &= q_1 a + q_2 b + q_3 a \\
 \Rightarrow q_2 &= \Lambda q_1 (a + a(b + aa)^* b) \\
 \Rightarrow q_1 &= (a + a(b + aa)^* b)^*
 \end{aligned}$$

Now:

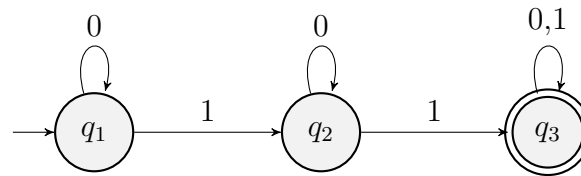
$$\begin{aligned}
 q_3 &= q_2 a \\
 &= q_1 a (b + aa)^* a \\
 &= (a + a(b + aa)^* b)^* a (b + aa)^* a \\
 &\Rightarrow \mathcal{A}
 \end{aligned}$$

1.5 Equivalence of Two Finite Automata

Two finite automata over Σ are equivalent if they accept the same set of strings over Σ .

Example 1.5.1. Consider the following two DFAs M and M' over $\{0,1\}$. Find out whether M and M' are equivalent.

Solution:



(q_i, q'_i)	(q_0, q'_0)	(q_3, q'_1)
(q_1, q_1)	(q_1, q_1)	(q_3, q_3)
(q_3, q_1)	(q_3, q_3)	(q_3, q_3)
(q_3, q_2)	(q_3, q_2)	(q_3, q_2)
(q_2, q_2)	(q_3, q_2)	(q_3, q_4)

The first element of the first column must be (q_1, q_1) , i.e., initial state of M and M' .

As we do not get a pair (q, q') where q is a final state and q' is a non-final state (or vice versa) at every row, we proceed until all the elements in the second and third column are also in the first column. Therefore, M and M' are equivalent.

Chapter 2

Minimization of Finite Automata

2.1 Minimization Definitions

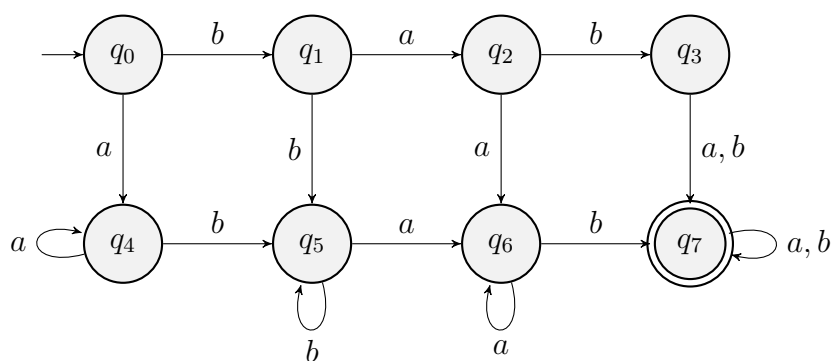
Definition 2.1.1. Two states q_1 and q_2 are equivalent if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non-final states for all $x \in \Sigma^*$.

Definition 2.1.2. Two states q_1 and q_2 are k -equivalent ($k \geq 0$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non-final states for all strings x of length k or less. In particular, any two final states are 0-equivalent and any two non-final states are also 0-equivalent.

2.2 Example: Construct the Minimum State Automaton

Example 2.2.1. Construct the minimum state automaton equivalent to the transition diagram given below:

Solution:



State/ Σ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_2
q_2	q_3	q_2
q_3	q_1	q_3
q_4	q_5	q_4
q_5	q_2	q_4
q_6	q_1	q_4

After minimization:

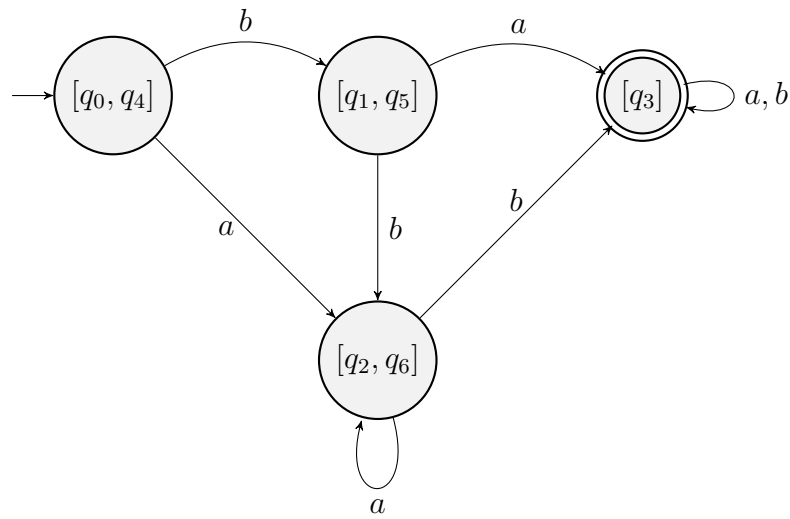
$$\Pi_0 = \{[q_3], [q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7]\}$$

$$\Pi_1 = \{[q_3], [q_0, q_2, q_4, q_5], [q_1, q_6, q_7]\}$$

$$\Pi_2 = \{[q_3], [q_0, q_2], [q_1, q_6], [q_4, q_5, q_7]\}$$

$$\Pi_3 = \{[q_3], [q_1, q_6], [q_2, q_5], [q_0, q_4]\}$$

The minimum state automaton:



Chapter 3

Formal Languages

3.1 Introduction

Definition 3.1.1 (Language). A language generated by a grammar G (denoted by $L(G)$) is defined as:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

If w contains non-terminals, it is called a sentential form.

If w does not contain non-terminals, it is called a sentence of G .

3.1.1 Grammar Definition

Definition 3.1.2. $L(G)$ is the set of all terminal strings derived from the start symbol S .

Two grammars are equivalent if they produce the same language.

3.2 Chomsky Classification of Languages

Definition 3.2.1 (Type-0). A type-0 grammar is any phrase-structured grammar without any restriction.

A production without any restriction is called a type-0 production.

Definition 3.2.2 (Type-1). A production of the form $\Phi A \Psi \rightarrow \Phi \alpha \Psi$ is called a type-1 production if $\alpha \neq \Lambda$.

In type-1 productions, erasing of A is not permitted.

Example 3.2.1. Examples of productions:

- (i) $AB \rightarrow \underline{ABC}$ is a type-1 production. The left context is A , right context is Λ .
- (ii) $A \rightarrow abA$ is a type-1 production. Here both the left and right context are Λ .

A grammar is called type-1 or context-sensitive if all its productions are type-1 productions. The production $S \rightarrow \Lambda$ is also allowed in a type-1 grammar, but in this case S does not appear on the right-hand side of any production.

The language generated by a type-1 grammar is called a type-1 or context-sensitive language.

In a context-sensitive grammar G , we allow $S \rightarrow \Lambda$ for including Λ in $L(G)$. Apart from $S \rightarrow \Lambda$, all the productions must be in the form where $\Lambda \in L(G)$. When $\Lambda \in L(G)$, we assume that S does not appear on the R.H.S. of any production.

Definition 3.2.3 (Type-2). A type-2 production is a production of the form $A \rightarrow \alpha$ where $A \in V_N$ and $\alpha \in (V_N \cup \Sigma)^*$. In other words, the L.H.S. has no left context and right context.

Example 3.2.2. Examples:

$$\circ S \rightarrow Aa, A \rightarrow a, B \rightarrow ab\epsilon, A \rightarrow \Lambda$$

A grammar is called a type-2 grammar if it contains only type-2 productions. It is also called a context-free grammar.

A language generated by a context-free grammar is called a type-2 language or context-free language.

Definition 3.2.4 (Type-3). A production of the form $A \rightarrow a$ or $A \rightarrow aB$ where $A, B \in V_N$ and $a \in \Sigma$, is called a type-3 production.

A grammar is called a type-3 or regular grammar if all its productions are type-3 productions. A production $S \rightarrow \Lambda$ is allowed in a type-3 grammar, but in this case S does not appear on the right-hand side of any production.

A language is regular if it can be described by a regular expression.

3.3 Relation Between Languages

$$L_{RL} \subseteq L_{CFL} \subseteq L_{CSL} \subseteq L_0$$

Two grammars of different types may generate the same language.

Operation	L_{RL}	L_{CFL}	L_{CSL}	L_0
Union	✓	✓	✓	✓
Concatenation	✓	✓	✓	✓
Transpose	✓	✓	✓	✓
Intersection	✓	×	✓	✓
Complement	✓	×	✓	✓
Kleene Star	✓	✓	✓	✓

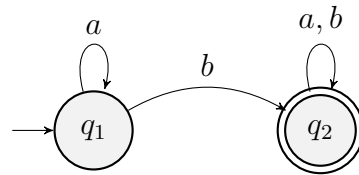
L_{CFL} is not closed under intersection, but the intersection of a context-free language and a regular language is context-free.

3.4 Construction of a Transition System Accepting a Regular Grammar

Example 3.4.1. Construct a regular grammar G generating the regular set represented by $P = a^*b(a+b)^*$.

Solution:

DFA corresponding to P is:



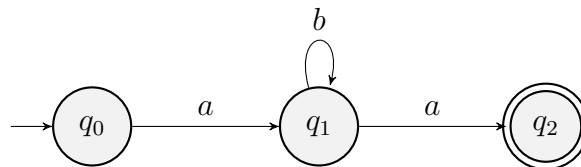
Regular grammar corresponding to P is:

$$\begin{aligned} A_0 &\rightarrow aA_0, & A_0 &\rightarrow bA_1, \\ A_1 &\rightarrow aA_1, & A_1 &\rightarrow bA_1, & A_1 &\rightarrow \Lambda \end{aligned}$$

Example 3.4.2. Construction of a transition system accepting a regular grammar G :

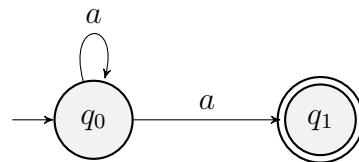
$$\begin{aligned} G &= (\{A_0, A_1\}, \{a, b\}, P, A_0) \\ P &= \{A_0 \rightarrow aA_1, A_1 \rightarrow bA_1, A_1 \rightarrow a, A_1 \rightarrow bA_1\} \end{aligned}$$

Solution:



Example 3.4.3. $S \rightarrow aS \mid a$

Solution:



Chapter 4

Context-Free Languages

4.1 Property of Languages That Are Regular

As a string is scanned left to right, the amount of memory that is required in order to determine at the end whether or not the string is in the language is the language is bounded and independent on the language, not the particular input string.

Regular languages with an infinite number of strings are represented by automata with cycles and regular expressions involving the Kleene star. It means they must have some repetitive structure.

Example 4.1.1. $L(G) = \{a^n \mid n \geq 1\}$ is a prime) is not regular.

$L = \{w \in \{a, b\}^* \mid w \text{ has an equal number of } a\text{'s and } b\text{'s}\}$ is not regular.

$L = a^*b^*$ is regular.

Every finite subset of a non-regular set is regular.

4.2 Ambiguity in Context-Free Grammars

Definition 4.2.1 (Ambiguity in CFG). A terminal string $w \in L(G)$ is ambiguous if there exists two or more derivation trees for w .

A context-free grammar G is ambiguous if there exists some $w \in L(G)$ which is ambiguous.

4.2.1 Construction of Reduced Grammars

For every CFG G , there exists a reduced grammar G' which is equivalent to G .

We construct the reduced grammar in two steps:

Step 1: We construct a grammar G_1 equivalent to the given grammar G so that every variable in G_1 derives some terminal string.

Step 2: We construct a grammar G' equivalent to G_1 so that every symbol in G' appears in some sentential form of G .

G' is the required reduced grammar.

Chapter 5

Elimination of Null Productions

Definition 5.0.1 (Null Production). A production of the form $A \rightarrow \Lambda$ where A is a variable, is called a null production.

A variable A in a context-free grammar is nullable if $A \Rightarrow^* \Lambda$.

If $G = (V_N, \Sigma, P, S)$ is a context-free grammar, then we can find a context-free grammar G_1 having no null or unit productions such that $L(G_1) = L(G) - \{\Lambda\}$.

5.1 Example: Find a Reduced Grammar Equivalent to the Given Grammar

Example 5.1.1. Find a reduced grammar equivalent to the grammar G where productions are:

$$\begin{aligned} S &\rightarrow AB \mid CA, & A &\rightarrow a, \\ B &\rightarrow BC \mid AB, & A &\rightarrow aB \mid \epsilon \mid b, & C &\rightarrow aB \mid b \end{aligned}$$

Solution:

Step-1: Construction of set W of all nullable variables:

$$\begin{aligned} W_0 &= \{S, A\}, & W_1 &= \{S, A\} = W_0 \\ S &\therefore U = \{S, A\} \end{aligned}$$

Step-2: Construction of P' :

$$S \rightarrow ASB \mid SB \mid AB \mid B$$

$$\begin{aligned} A &\rightarrow aAS \mid aS \mid aA \mid AB \mid B \mid a \\ B &\rightarrow SbS \mid Sb \mid bS \mid b \mid Aa \mid a \end{aligned}$$

Example 5.1.2. Remove left recursion from the following grammar:

$$S \rightarrow Aa \mid Aa\epsilon \mid Aa_1 \dots Aa_n \mid B_n$$

then

$$\begin{aligned} P_1 &= \{A \rightarrow BY \mid B_1Y_1 \mid \dots \mid B_nY_n\} \\ A &\rightarrow B_1 \mid B_2 \mid \dots \mid B_n \mid \Lambda \end{aligned}$$

5.2 Lemma 1: Deleting a Variable B Appearing as the First Symbol on the R.H.S.

Lemma 5.2.1. Let $P = \{A \rightarrow BY, B \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n\}$, then

$$P_1 = \{A \rightarrow \beta_1 Y \mid \beta_2 Y_1 \mid \dots \mid \beta_n\}$$

5.3 Lemma 2: Eliminating Left Recursion

Lemma 5.3.1. Let $P = \{A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m\}$ where no β_i 's production $\alpha_i \in \alpha \in (\Sigma \cup V_N)^*$.

$$P_1 = \{A \rightarrow \beta_1 Z \mid \beta_2 Z \mid \dots \mid \beta_m Z, Z \rightarrow \alpha_1 \mid \alpha_2 Z_1 \mid \dots \mid \alpha_n Z, Z \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n Z_n\} \dots$$

Chapter 6

Greibach Normal Form

6.1 Definition

Definition 6.1.1 (Greibach Normal Form). A context-free grammar generating the set accepted by a pushdown automaton is in Greibach Normal Form.

A context-free grammar is in Greibach Normal Form if every production is of the form:

$$A \rightarrow a\alpha, \quad \text{where } \alpha \in V_N^* \text{ and } a \in \Sigma$$

(or may be Λ), and $S \rightarrow \Lambda$ is an Λ and where S does not appear on R .

Chapter 7

Chomsky Normal Form for Context-Free Grammars

Definition 7.0.1 (Chomsky Normal Form). A context-free grammar G is in Chomsky normal form if every production is of the form:

$$A \rightarrow a, \quad \text{or} \quad A \rightarrow BC,$$

and $S \rightarrow \Lambda$ is in G if $\Lambda \in L(G)$.

When Λ is in $L(G)$, we assume that S does not appear on the R.H.S of any production.

7.1 Reduction to Chomsky Normal Form

Step 1: Elimination of null productions and unit productions.

Step 2: Elimination of terminals on R.H.S.

We define $G_2 = (V_N, \Sigma, P_2, S)$ as follows:

- (i) All productions in P_1 are added to P_2 if they are in the required form. All the variables in V_1 are added to V_N .
- (ii) If $S \rightarrow aD$ gives rise to $S \rightarrow CaD$ and $Ca \rightarrow b$
- (iii) $A \rightarrow bAB$ gives rise to $A \rightarrow CbAB$ and $Cb \rightarrow b$
- (iv) $A \rightarrow bAB$ gives rise to $A \rightarrow CbAB$ and $Cb \rightarrow b$
- (v) $V_N = \{S, A, B, D, Ca, Cb\}$

Step 3: $P_1 = \{S \rightarrow CaAD, A \rightarrow CaCb \mid CaCb\}$

$$Ca \rightarrow a, Cb \rightarrow b$$

$A \rightarrow CaB, B \rightarrow b, D \rightarrow d, Ca \rightarrow a, Cb \rightarrow b$ are added
 $S \rightarrow CaAD$ is replaced by $S \rightarrow CaCb$ and $C \rightarrow AD$

$A \rightarrow CbAB$ is replaced by $A \rightarrow CaC_1$ and $C_2 \rightarrow AB$

Now, $G_n = (\{S, A, B, D, Ca, Cb, C_1, C_2\}, \{a, b, d\}, P_3, S)$ where

$$P_3 = \{S \rightarrow CaC_1, A \rightarrow CaB \mid CaCb \mid CaC_2\}, \dots$$

Example 7.1.1. Example: Remove left recursion from the following grammar:

$$A \rightarrow aBD \mid bDB \mid \epsilon, \quad A \rightarrow AB \mid AD$$

Solution:

$$\begin{aligned} A &\rightarrow aBD \mid bDB \mid \epsilon, & A &\rightarrow aBDZ \mid bDBZ \mid \epsilon Z \\ Z &\rightarrow BZ \mid DZ \mid B \mid D \end{aligned}$$

Example 7.1.2. Remove A -productions from the following grammar:

$$\begin{aligned} S &\rightarrow ASB \mid A, & A &\rightarrow aAS \mid aB \mid AB \mid B \mid a \\ B &\rightarrow Sbs \mid Sb \mid bS \mid b \mid Aa \mid a \end{aligned}$$

Solution: Step-1: Construction of set W of all nullable variables:

$$\begin{aligned} W &= \{S, A\}, & W_1 &= \{S, A\} = W_0 \\ \therefore U &= \{S, A\} \end{aligned}$$

Step-2: Construction of P' :

$$\begin{aligned} S &\rightarrow ASB \mid AB \mid B \\ A &\rightarrow a \end{aligned}$$

7.2 Remove A -production from the following grammar

Example 7.2.1.

$$\begin{aligned} A &\rightarrow aBDI \mid bDBZ \mid \epsilon \\ Z &\rightarrow BZ \mid DZ \mid B \mid D \end{aligned}$$

Solution:

$$\begin{aligned} A &\rightarrow aBDIBDBZ \mid \epsilon, & A &\rightarrow aBDZ \mid bDBZ \mid Z \\ Z &\rightarrow BZ \mid DZ \mid B \mid D \end{aligned}$$

Step-1: Construction of set W of all nullable variables:

$$\begin{aligned} W_0 &= \{S, A\}, & W_1 &= W_2(\epsilon S) = W_0(\epsilon S) \cup \phi = W_0(\epsilon S) \\ \text{Hence } W(\epsilon S) &= \{S\} \end{aligned}$$

Similarly:

$$\begin{aligned} W(A) &= \{A\}, & W(E) &= \{E\} \\ W_0(B) &= \{B\}, & W(B) &= \{B\} \\ W_0(C) &= \{C\}, & W_1(E) &= \{B, C\} \cup \{E\} = \{B, C, E\} \\ W_2(B) &= \{B, C, DE\} \\ W_3(B) &= \{B, CDE\} \end{aligned}$$

Therefore $W(B) = \{B, C, DE\}$.

Step-2: The productions in G_1 are:

$$\begin{aligned} S &\rightarrow AB, & A &\rightarrow a, & E &\rightarrow a, \\ B &\rightarrow aD \mid D & \text{ or } & B &\rightarrow CaD \text{ and } C &\rightarrow AD \end{aligned}$$

$S \rightarrow CaAD$ is replaced by $S \rightarrow CaC_1$ and $C \rightarrow AD \dots$

Now, $G_n = (\{S, A, B, D, Ca, Cb, C_1, C_2\}, \{a, b, d\}, P_3, S)$ where

$$P_3 = \{S \rightarrow CaC_1, A \rightarrow CaB \mid CaCb \mid Ca\}$$

Note: $L = \{a^n b^c \mid n \geq 1\}$ is not context-free but context-sensitive.

Also, $L = \{a^n \mid p \text{ is a prime}\}$ is not context-free.

Chapter 8

Pushdown Automata

8.1 Introduction

Definition 8.1.1 (Pushdown Automaton). Each context-free language is accepted by some pushdown automaton.

If a language is accepted by a pushdown automaton, it is a context-free language.

8.1.1 Pushdown Automaton for $(a^n b^n \mid n \geq 1)$ (Initial Pushdown Symbol)

Let $A = (Q, \Sigma, \Gamma, \delta, q_0, \Sigma_0, F)$

$$\delta \Rightarrow Q \times (\Sigma \cup \{\Lambda\}) \times \Gamma \rightarrow Q \times \Gamma^*$$

$$\text{where: } Q = \{q_0, q_1, q_2\}, \quad \Sigma = \{a, b\}, \quad \Gamma = \{a, Z_0\},$$

$$F = \{q_2\} \text{ and}$$

S is given by:

$$\delta(q_0, a, Z_0) = (q_0, aZ_0)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \Lambda)$$

$$\delta(q_1, b, a) = (q_1, \Lambda)$$

$$\delta(q_1, \Lambda, Z_0) = (q_2, \Lambda)$$

8.2 Infinite Context-Free Languages

Infinite context-free languages display periodicity—of a somewhat subtler form than do regular languages.

8.3 Determinism and Parsing

A pushdown automaton is not of immediate use in parsing because it's a nondeterministic device.

A pushdown automaton M is deterministic if for each configuration, there is at most one configuration that can succeed it in a computation by M .

Deterministic Context-Free Languages are essentially those that are accepted by deterministic pushdown automaton.

There are some context-free languages that cannot be accepted by deterministic pushdown automata.

The machine to accept the language $\{wcw^R \mid w \in \{a, b\}^*\}$ is deterministic. We use the machine to accept $\{w \in \{a, b\}^* \mid w = w^R\}$ is nondeterministic.

The class of deterministic context-free languages is closed under complement.

Nondeterminism is more powerful than determinism in the context of pushdown automaton.

Chapter 9

Turing Machines and Recursive Enumerable Languages

9.1 Turing Machines

Definition 9.1.1 (Turing Machine). A Turing machine consists of a finite control, a tape, and a head that can be used for reading or writing on that tape.

The Turing machines can form a stable and maximal class of computational devices, in terms of the computations they can perform.

A Turing machine M does not accept w if the machine either halts in a non-accepting state or does not halt.

A general purpose Turing machine is usually called a universal Turing machine which is powerful enough to simulate the behavior of any computer including any Turing machine itself.

If we increase the number of heads or make the tape two or three dimensional, the Turing machine will almost speedup the operation of the machine, but will not increase the computing power of doing something more than basic model.

9.2 Linear Bounded Automaton

Definition 9.2.1 (Linear Bounded Automaton). A Linear Bounded Automaton is a nondeterministic Turing machine which has a single tape whose length is not infinite but bounded by a linear function of the length of the input string.

The set of languages accepted by nondeterministic Turing machine is same as the set of languages accepted by deterministic Turing machine.

In linear bounded automaton, the length of the tape is restricted.

If L is a context-sensitive language, then L is accepted by a Linear Bounded Automaton (LBA). The converse is also true.

Chapter 10

Recursive and Recursive Enumerable Languages

10.1 Definitions

Definition 10.1.1 (Recursive Enumerable Language). A language is recursive enumerable if and only if there is a Turing machine M that semidecides it.

OR

A recursive enumerable set is a set X for which we have a procedure to determine whether a given element belongs to X or not.

Every recursive language is recursive enumerable, but not the two terms are not the same.

Recursive Language $\subseteq$ Recursive Enumerable Language

The class of recursive languages is a proper subset of the class of recursively enumerable languages.

If a L is a recursive language, then its complement L^c is also recursive.

A language is recursive if and only if both it and its complement are recursively enumerable.

The class of recursively enumerable languages is not closed under complement.

A language is generated by a grammar if and only if it is recursively enumerable.

Recursive Enumerable Language = Type-0 Language.

Chapter 11

Undecidability

11.1 Halting Problem

Definition 11.1.1 (Halting Problem). Halting Machine takes as input an encoding of a Turing machine $e(M)$ and an encoding of an input string $e(w)$, and returns "yes" if M halts on w and "no" if M does not halt on w .

The halting problem is undecidable:

- There exists no Turing machine that decides it.
- There is no Turing machine that halts on all inputs, and always say "yes" if M halts on w , and always say "no" if M does not halt on w .

The following problems about Turing machines are undecidable:

- (i) Given a Turing machine M , does M halt on the empty tape?
- (ii) Given two Turing machines M_1 and M_2 , do they have the same input strings?
- (iii) Given a Turing machine M , is the language that M semidecides regular? Is it context-free? Is it recursive?

11.2 Reduction

To solve a problem B as undecidable:

- (i) Start with an instance of a known undecidable problem.
- (ii) Create an instance of problem B , such that the answer to the instance of problem B gives the answer to the undecidable problem.
- (iii) If we could solve problem B , we could solve the halting problem.
- (iv) Thus, problem B must be undecidable.

11.3 Unsolvability Problems about Grammars

If we can reduce problem A to problem B , and problem A is undecidable, then:

it tells nothing about problem B .

If we can reduce problem B to problem A , and problem A is undecidable, then:

B tells nothing about problem B .

Each of the following problems is undecidable:

- (1) Given a grammar G and a string w , is $w \in L(G)$?
- (2) Given a CFG G , is $L(G) = \Sigma^*$?
- (3) Given two CFGs G_1 and G_2 , is $L(G_1) = L(G_2)$?
- (4) Given two PDA M_1 and M_2 , is $L(M_1) = L(M_2)$?

However, there are some problems about CFGs that are undecidable:

- (1) Given any CFG G , is $L(G) = \emptyset$?
- (2) Given any two CFGs G_1 and G_2 , is $L(G_1) = L(G_2)$?
- (3) Given two PDA M_1 and M_2 , is $L(M_1) = L(M_2)$?
- (4) Given a PDA M , find an equivalent PDA with the smallest possible number of states.
- (5) Tiling problem is undecidable.
- (6) Ambiguity problem for CFGs is undecidable.

The following questions about Context-Free Grammars are decidable:

- (1) Given a C.F.G. G and string w , is $w \in L(G)$?
- (2) Given a CFG G , is $w \wedge \in L(G)$?

Chapter 12

Some Important Results

12.1 Regular Expression Results

- (1) Regular Expression for all strings over $\{0, 1\}$:
 - have an even number of 1's: $0^*(10^*10^*)^*$
 - have no occurrences of 00: $1^*(01^*)^*(01^*)$
 - have exactly one occurrence of 00: $1^*(01^*)^*00(1 + 01)^*1^*$
 - do not contain 01: 1^*0^*
- (2) $L = \{a^n b^c \mid n \geq 1\}$ is not context-free, but context-sensitive.
- (3) $L = \{a^n \mid p \text{ is a prime}\}$ is not a context-free language.
- (4) $L = \{a^n l^n \mid n \geq 1\}$ is not regular, but context-free.
- (5) $L = \{a^n \mid n \geq 0\}$ is not context-free.
- (6) $L = \{a^n \mid n \geq 1\}$ is regular.
- (7) $L = \{babaabaaa \dots ba^{2^n} a^n \mid n \geq 1\}$ is not context-free.
- (8) $L = \{wxw^T \mid w \in \{a, b\}^*\}$ is regular.
- (9) $L = \{ww^T \mid w \in \{a, b\}^*\}$ is context-free.

Chapter 13

NP Completeness

13.1 Tractable Problems

Definition 13.1.1 (Tractable Problems). The set of all the problems that can be solved within polynomial amount of time using deterministic machines is of type $O(n^k)$.

13.2 Intractable Problems

Definition 13.2.1 (Intractable Problems). The set of all problems that can't be solved within polynomial amount of time using deterministic machines is of type $O(c^n)$.

13.3 P-Class Problems

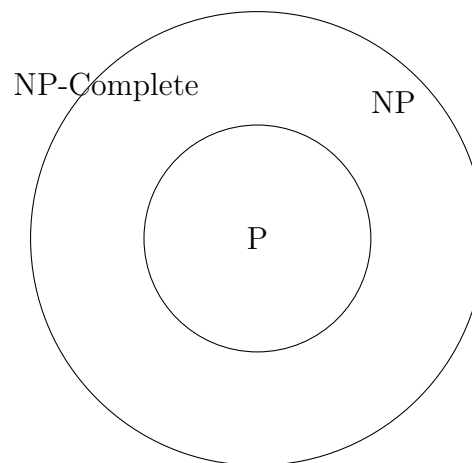
The set of problems that can be solved in rational polynomial amount of time by deterministic machines, is called the P-class or polynomial problems.

13.4 NP-Class Problems

The set of problems that can be solved in polynomial time using non-deterministic machines.

$NP = \text{Non-deterministic Polynomial Time}$

$NP \neq \text{Non-polynomial}$



$$P \subseteq NP$$

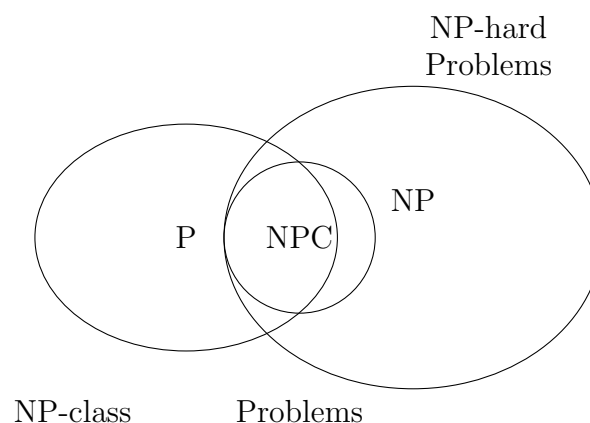
The class NP consists of those problems that are verifiable in polynomial time.

13.5 NP-Hard Problems

These problems are at least as hard as the hardest problems in NP, but not necessarily in NP.

The problems to which all NP-class problems are reducible in polynomial time, are known as NP-hard class of problems.

13.5.1 P-class Problems



13.6 NP-Complete Problems

NP-Complete problems are those NP-hard problems which are within NP.

These are not of the hardest problems in NP, i.e., in real sense these are the ones that most likely not to be in P.

NP-Complete problems are solvable using heuristic.

It is not known whether NP problems are tractable or intractable.

If a problem in a class of NP-Complete can be solved in polynomial time, then all the problems in that class can be solved in a polynomial time using the same algorithm.

If a deterministic polynomial time algorithm is found for an NP-Complete problem, then every problem in NP-class can be solved in deterministic polynomial time.

The class of all polynomially decidable languages denoted by P is closed under union, intersection, concatenation, complement, and Kleene Star.

13.7 Examples of NP-Complete Problems

- (1) Bounded Tiling
- (2) 3-Satisfiability / Satisfiability (SAT)
- (3) MAX-SAT
- (4) Hamiltonian Cycle
- (5) Traveling Salesman Problem
- (6) Exact Cover and Node Cover
- (7) Independent Set
- (8) 0-1 Knapsack
- (9) Bin Packing
- (10) Printed Circuit Board (PCB)
- (11) CLIQUE

www.ankurgupta.net